LAB- 3 Short-time Fourier Transform (STFT) and Correlation

Part 1

The first part of this lab focuses on understanding STFT, spectrograms, and their real-world applications. In previous labs, you worked with FFTs in your code, generated, and analyzed spectograms. In this lab, you will:

1) Generate and analyse time-varying signals
2) Compute, visualize, and analyze spectrograms
3) Compare different window functions and segment lengths
4) Apply the STFT to real-time audio signals

The Short-Time Fourier Transform (STFT) is used in analysis by computing the Fourier Transform over short, overlapping time segments. It divides the signal into smaller segments (windows) and computes the Fourier Transform on each. This gives a time-frequency representation (like a spectrogram), where we can see how frequencies evolve over time.

STFT has many applications and is used in many fields, such as musical analysis, speech processing, and biomedical applications. In your report, give three examples of some applications of the STFT.

Experiment 1

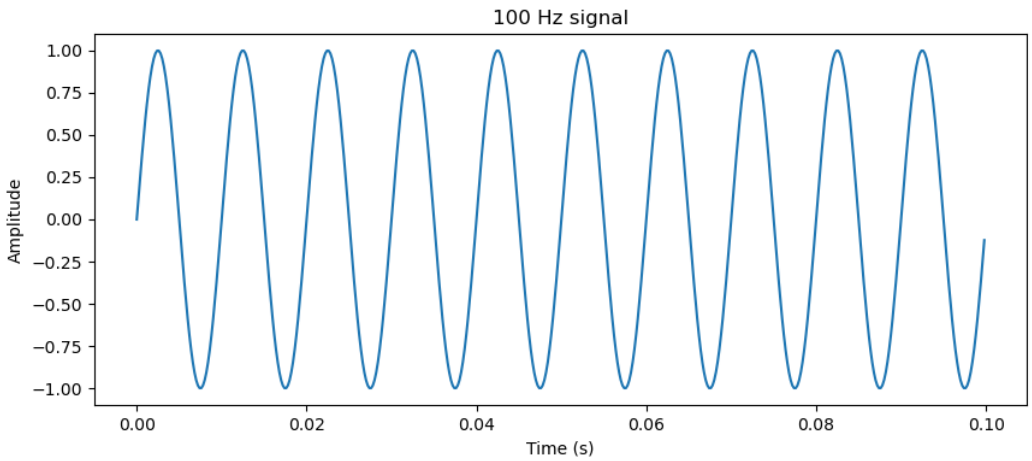
1) Create a new Python file.
2) Install the librosa library using the terminal. (Refer to lab 1 on how to install libraries.)
3) Import the following libraries.

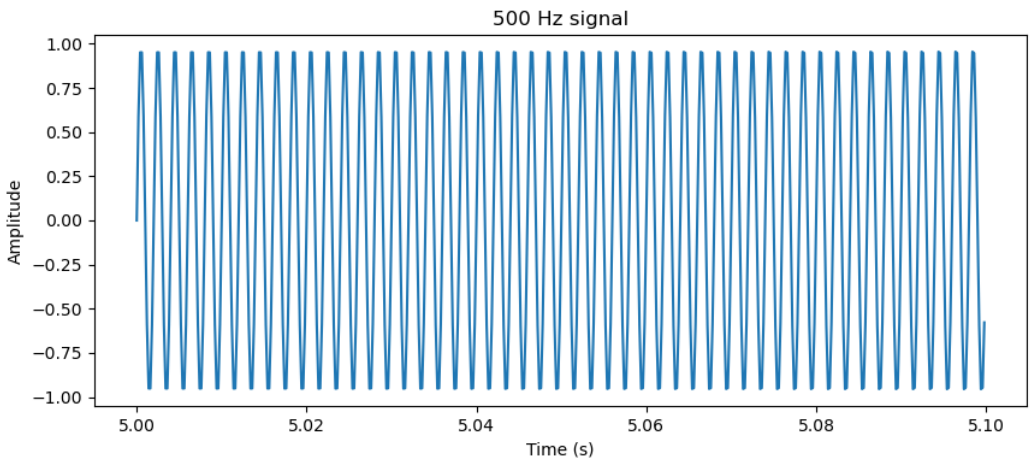
```
TEST.py
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as signal
import librosa
import librosa.display
import sounddevice as sd
import scipy.io.wavfile as wav
```

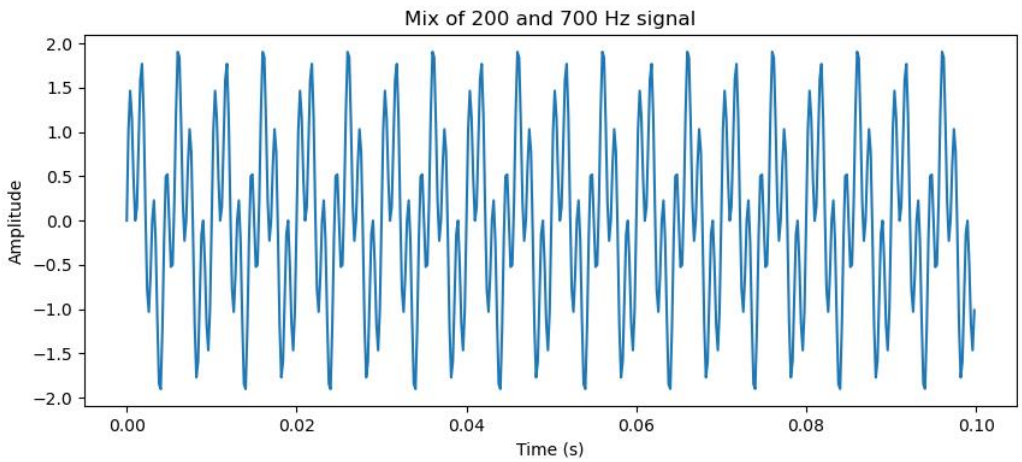
4) Generate a time-varying signal with a period of 10s at a sampling rate of 5kHz.
5) The time-varying signal should be a mix of three signals, with the first half of the signal being of a low frequency (100Hz), the second half being of a higher frequency (500Hz), and finally a third signal which consists of a mix of frequencies (200 and 700Hz) occurring throughout the signal.
6) Combine these signals using np.concatenate(()).
7) Plot the signal and attach the figure to your report.
8) Compute the STFT of the signal using the following piece of code.
   Line 29 is used to obtain the STFT. In this line, 'frequencies' creates an array of frequency values on the Y-axis and 'times' an array of time values (X-axis). 'Zxx' gives the complex STFT matrix containing time-frequency data. 'x' represents the signal, and 'nperseg' represents the number of samples per segment (window size). 'plt.colormesh' creates a spectrogram with color representing amplitude.

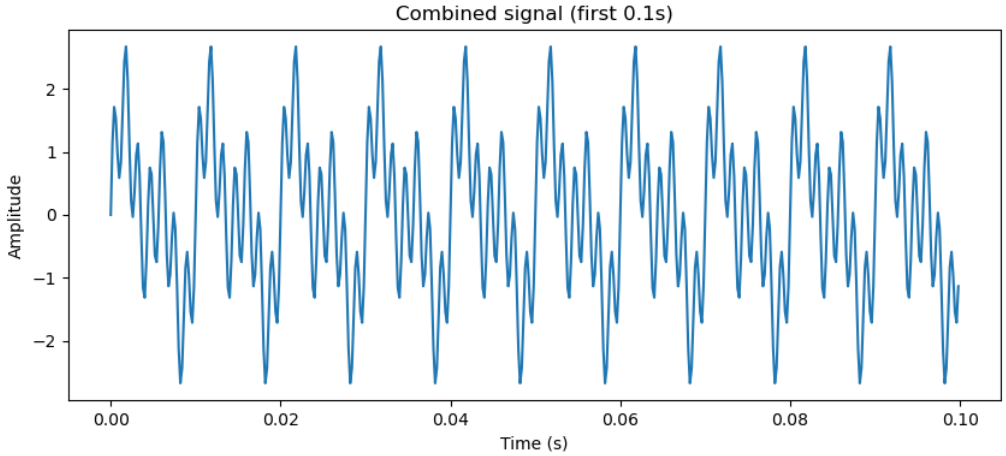
```python
# Compute STFT
frequencies, times, Zxx = signal.stft(x, fs, nperseg=256)

# Plot Spectrogram
plt.figure(figsize=(10, 5))
plt.pcolormesh( *args: times, frequencies, np.abs(Zxx), shading='gouraud')
plt.colorbar(label="Magnitude")
plt.ylabel("Frequency (Hz)")
plt.xlabel("Time (s)")
plt.title("STFT Spectrogram")
plt.show()
```

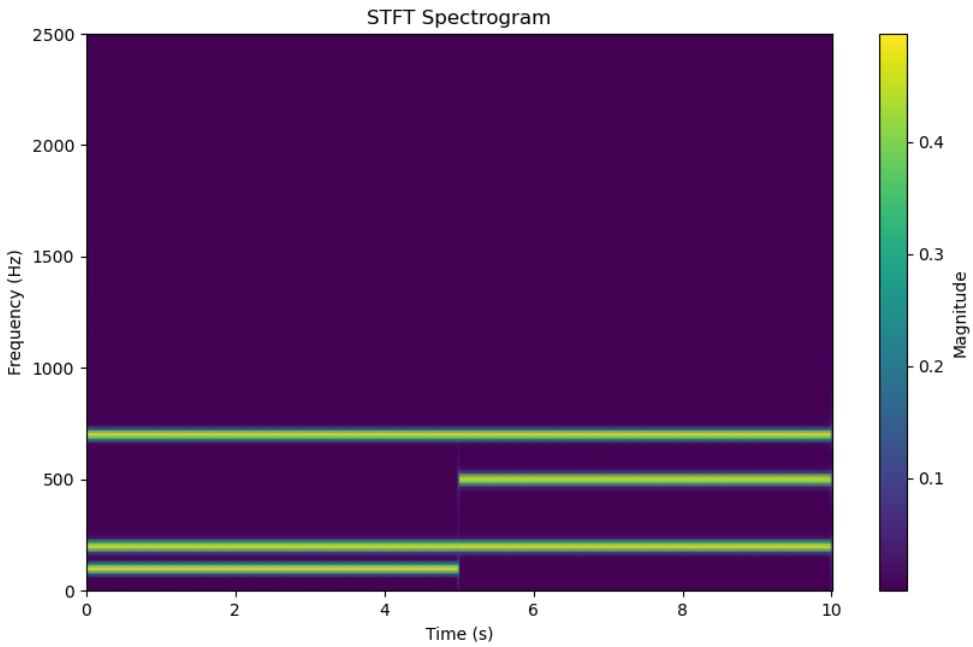
9) Attach the plot to the report.











<u>Assignment 1 –</u> Run the code from experiment 1, attach your plots, and answer the following questions.

1) What does the spectrogram tell us about the signal?

It shows that the frequencies present at time intervals are changing showing the signal isn't stationary.

2) Are the frequency components stable or changing over time?

They are changing over time

3) In your spectrogram, you should observe some bright lines. Why do we see horizontal bright lines at specific frequencies?

For the concatenated signal, the STFT slides a window across the signal and computes the FFT at each position. For a single frequency sinusoidal wave, all the energy is concentrated within a single frequency bin. Frequencies that appear dark are caused by zero-padding.

4) Increase the window size gradually. What changes do you observe?

The magnitude line gets thinner

5) Now, decrease the window size and state your observations.

The magnitude line gets thicker.

6) What conclusion can you draw about the relationship between the window size and the STFT?

As you increase the window size, the FFT resolution results in more accurate frequency bin. Decreasing window size decreases the resolution of frequency bins.

Experiment 2

In this experiment, we will look at analyzing real-time speech using the STFT. Try as much as possible to be in an area with minimum noise to obtain the best results while you do the recoridngs. You can decide to create a new Python file for this experiment, but make sure you import the necessary libraries. Take the time to figure out what each line of code does on your own.

1) Record an audio of you speaking for a duration of 5s. You can change the duration to about 10s if you want.

```
# Record audio (5 seconds)
fs = 16000  # Standard speech sampling rate
duration = 5  # seconds
print("Recording...")
audio = sd.rec(int(duration * fs), samplerate=fs, channels=1, dtype='float32')
sd.wait()
print("Recording complete!")
```

2) Save the recorded audio using;

```
# Save the audio
wav.write( filename: "speech_recorded.wav", fs, (audio * 32767).astype(np.int16))
```

3) Compute the STFT with the following lines of code.

```
# Load and Compute STFT
y, sr = librosa.load( path: "speech_recorded.wav", sr=fs)
frequencies, times, Zxx = signal.stft(y, sr, nperseg=512)
```

4) Plot the frequency response as follows.

```
# Plot the STFT Spectrogram
plt.figure(figsize=(10, 5))
plt.pcolormesh( *args: times, frequencies, np.abs(Zxx), shading='gouraud')
plt.colorbar(label="Magnitude")
plt.ylabel("Frequency (Hz)")
plt.xlabel("Time (s)")
plt.title("Speech STFT Spectrogram")
plt.show()
```

Assignment 2 – Run the code from experiment 2, attach your plots, and answer the following questions.

1) What frequency range does your speech recording cover?

100-500 Hz

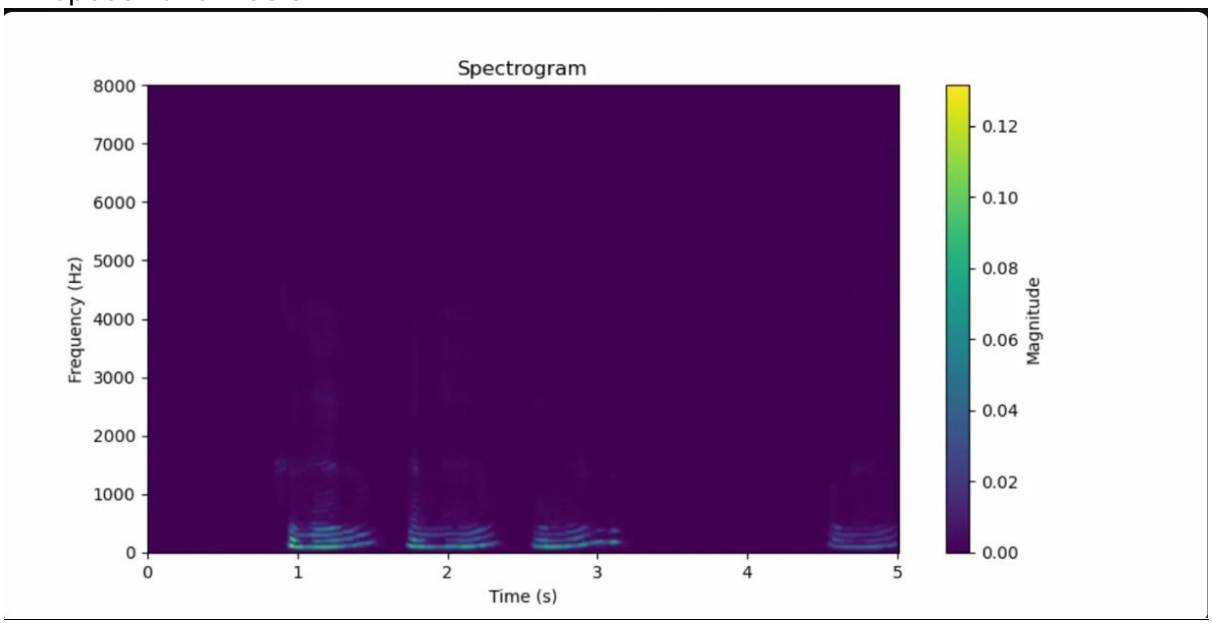
2) How does time resolution vs. frequency resolution affect the spectogram?

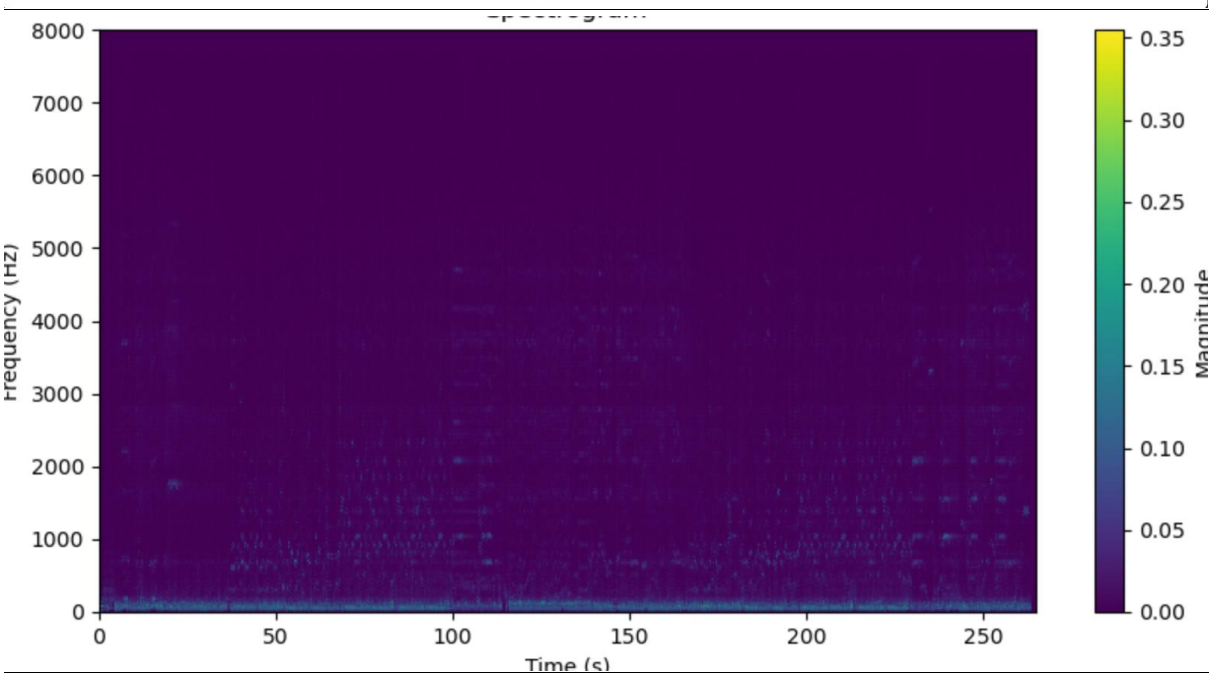
A high time resolution clearly identifies start and end of events but blurs the ability to distinguish distinct frequencies. A high frequency resolution identifies distinct frequencies but start and end time of events are hard to identify.

3) How can background noise be identified in the spectogram?

Low intensity and consistent horizontal lines around frequencies range are present where no significant events occur.

4) What would happen if you applied this code to a recording of a piece of music? Include your plot and compare the differences between the two spectograms of speech and music.





The music has energy spread out continuously through the 0-250 frequency range with energy peaking around 4000-5000 Hz. Music has a fundamentally more complex frequency range than a voice recording.

5) How can we use spectograms for speaker recognition?

Every person has a unique harmonic frequency, so it is possible to find the frequency range for a specific person.

Part 2

The second part of this lab focuses on correlation and autocorrelation and how they can be used in DSP.

Correlation measures the similarity between two signals as a function of time shift. It helps in detecting patterns, delays, or similarities between two signals. There is a difference between convolution and correlation, though they might be mathematically similar. Find out this difference and include it in your report.

Cross-correlation measures the similarity between two different signals and is used in time delay estimation, signal matching, and feature detection.

Autocorrelation measures the similarity of a signal with a delayed version of itself. It can be used for pitch detection, periodicity analysis, and noise reduction.

You can refer to lecture 7 for more details on the two types.

Experiment 3

We will use signal energy to classify audio frames as voiced (high energy) or unvoiced (low energy). This is commonly used in speech processing. The energy of a discrete signal x[n] over a frame of length N is given as:

$$E_s = \sum_{n=0}^{N-1} x[n]^2$$

1) Download the test_vector.wav file and add it to your project path.
2) Import the necessary libraries you will need

```
import numpy as np
import matplotlib.pyplot as plt
import soundfile as sf
```

3) Read the audio file

```
# **Read the Audio File**
filename = "test_vector.wav"
signal, sample_rate = sf.read(filename)
```

4) This next piece of code is to ensure we are working with mono for easier computation. This is done by just taking the mean of the signal. Next, we set the frame size using the sample_rate. We split the signal into 20ms frames with 50% overlap.

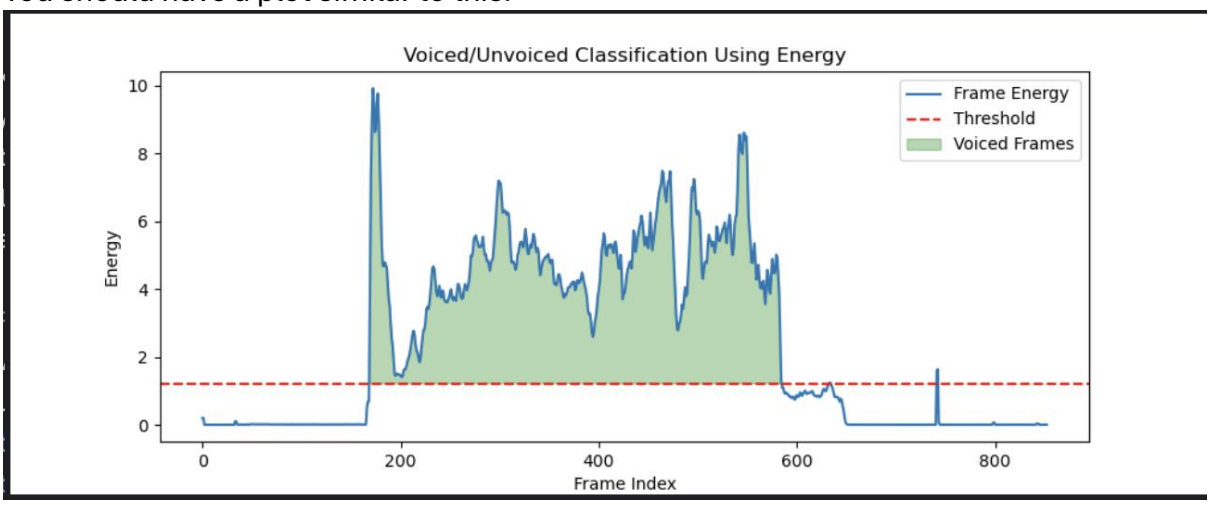
```
# **Convert to Mono if Stereo**
if len(signal.shape) > 1:
    signal = np.mean(signal, axis=1)

# **Frame Settings**
FRAME_SIZE = int(0.02 * sample_rate)  # 20ms per frame
HOP_SIZE = FRAME_SIZE // 2  # 50% overlap
```

5) The next step involves computing the energy per frame. Use the formula given above and with a for loop and numpy, compute the energy for each frame.
6) Now, set a threshold to determine if the frame is voiced or unvoiced. If the energy is greater than the threshold, classify it as voiced.

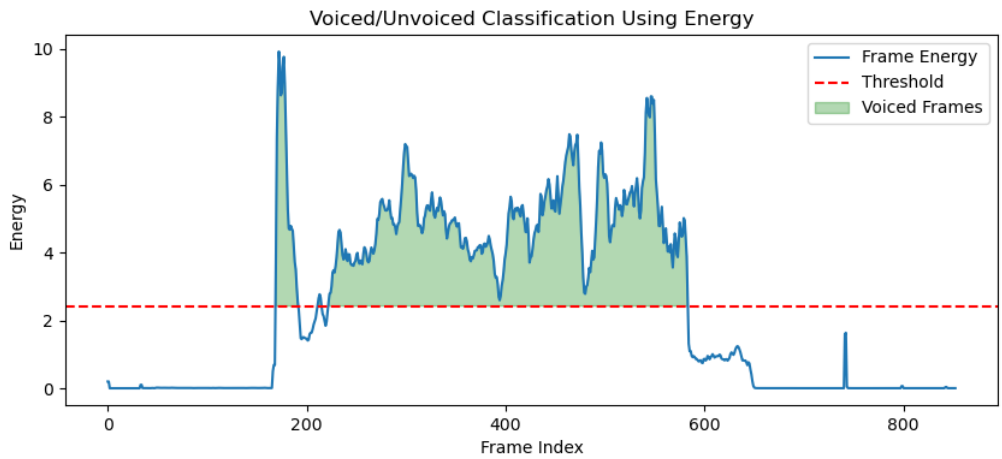
7) Generate a plot of the energy with Voiced/Unvoiced Classification using the code below as a template.

```python
# **Plot Energy with Voiced/Unvoiced Classification**
plt.figure(figsize=(10, 4))
plt.plot( *args: energy_values, label="Frame Energy")
plt.axhline(threshold, color="red", linestyle="--", label="Threshold")
plt.fill_between(range(num_frames), energy_values, threshold, where=voiced_labels,
                 color='green', alpha=0.3, label="Voiced Frames")
plt.xlabel("Frame Index")
plt.ylabel("Energy")
plt.title("Voiced/Unvoiced Classification Using Energy")
plt.legend()
plt.show()

# **Print Results**
num_voiced = np.sum(voiced_labels)
num_unvoiced = len(voiced_labels) - num_voiced
print(f"Total Frames: {num_frames}")
print(f"Voiced Frames: {num_voiced} ({num_voiced/num_frames:.2%})")
print(f"Unvoiced Frames: {num_unvoiced} ({num_unvoiced/num_frames:.2%})")
print(f"Threshold Used: {threshold:.5f}")
```

8) You should have a plot similar to this.



<u>Assignment 3:</u> Run the code from the above experiment, attach your plot, and answer the following questions.



1) What value did you use as the threshold and why?

The mean of the signal as the threshold assumes the middle of the voiced and unvoiced values making it a simple starting point.

2) What are the potential limitations to using energy alone for voiced/unvoiced classification?
3) What other methods can be used?
4) What would happen if the hop size were increased to 100% (no overlap)?
5) What would happen if we used a smaller frame size (10ms instead of 20ms)?
6) Could this approach be used in real-time speech processing? What modifications would be needed?
7) Implement (6) in code and attach it to the report.

<u>Experiment 4</u>

This experiment illustrates autocorrelation (self-similarity) and cross-correlation (similarity between two signals).

1) Import the necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
import soundfile as sf
from scipy.signal import correlate
```

2) Generate and plot the following signals

```
# **Generate Test Signals**
fs = 1000  # Sampling frequency (Hz)
t = np.arange(0, 1, 1/fs)  # 1 second time vector

# **Signal 1: Pure Sine Wave**
f1 = 5  # Frequency in Hz
signal1 = np.sin(2 * np.pi * f1 * t)

# **Signal 2: Same Sine Wave but Delayed**
delay = 50  # Delay in samples
signal2 = np.roll(signal1, delay)  # Shift signal by 50 samples

# **Signal 3: Random Noise**
signal3 = np.random.randn(len(t))
```

3) We will now define the autocorrelation and cross-correlation functions using numpy and scipy functions as follows. The sine wave autocorrelation should show a periodic pattern, and the noise autocorrelation should look more

random.

```python
def compute_autocorrelation(signal):  2 usages
    """Compute and normalize the autocorrelation of a signal."""
    autocorr = correlate(signal, signal, mode="full")
    autocorr = autocorr / np.max(autocorr)  # Normalize
    return autocorr[len(autocorr)//2:]  # Keep positive lags

# **Compute Autocorrelation**
autocorr1 = compute_autocorrelation(signal1)
autocorr3 = compute_autocorrelation(signal3)

# **Plot Autocorrelation**
plt.figure(figsize=(10, 4))
plt.plot( *args: autocorr1, label="Autocorrelation of Sine Wave")
plt.plot( *args: autocorr3, label="Autocorrelation of Noise", linestyle="--")
plt.xlabel("Lag")
plt.ylabel("Normalized Correlation")
plt.title("Autocorrelation of Signals")
plt.legend()
plt.show()
```

```python
def compute_cross_correlation(signal_a, signal_b):  1 usage
    """Compute and normalize the cross-correlation between two signals."""
    cross_corr = correlate(signal_a, signal_b, mode="full")
    cross_corr = cross_corr / np.max(cross_corr)  # Normalize
    lags = np.arange(-len(signal_a) + 1, len(signal_a))
    return lags, cross_corr

# **Compute Cross-Correlation Between Two Sine Waves**
lags, crosscorr = compute_cross_correlation(signal1, signal2)

# **Plot Cross-Correlation**
plt.figure(figsize=(10, 4))
plt.plot( *args: lags, crosscorr, label="Cross-Correlation (Signal 1 vs. Signal 2)")
plt.axvline(delay, color="red", linestyle="--", label=f"Expected Delay = {delay} samples")
plt.xlabel("Lag (Samples)")
plt.ylabel("Normalized Correlation")
plt.title("Cross-Correlation Between Two Signals")
plt.legend()
plt.show()
```

Assignment 4 – Run the code, attach your plots, and answer the following questions.

1) How can cross-correlation help detect signal delay?
2) What does periodicity in autocorrelation represent?
3) Would cross-correlation work if the signals had different frequencies?
4) Redefine the functions to perform autocorrelation and cross-correlation manually without the use of the correlate function. Use the formula from the slides to guide you.
5) Compare your results obtained with the original plots. Any similarities or differences?

Assignment 5

1) Write code to record a speech signal and use an autocorrelation function to detect the pitch of the signal.
2) Modify the code to work for real-time speech.

Links

1) STFT - https://en.wikipedia.org/wiki/Short-time_Fourier_transform
2) np.concatenate - https://numpy.org/doc/stable/reference/generated/numpy.concatenate.html
3) Correlation - https://www.dspguide.com/ch7/3.htm
4) np.correlate - https://numpy.org/doc/stable/reference/generated/numpy.correlate